

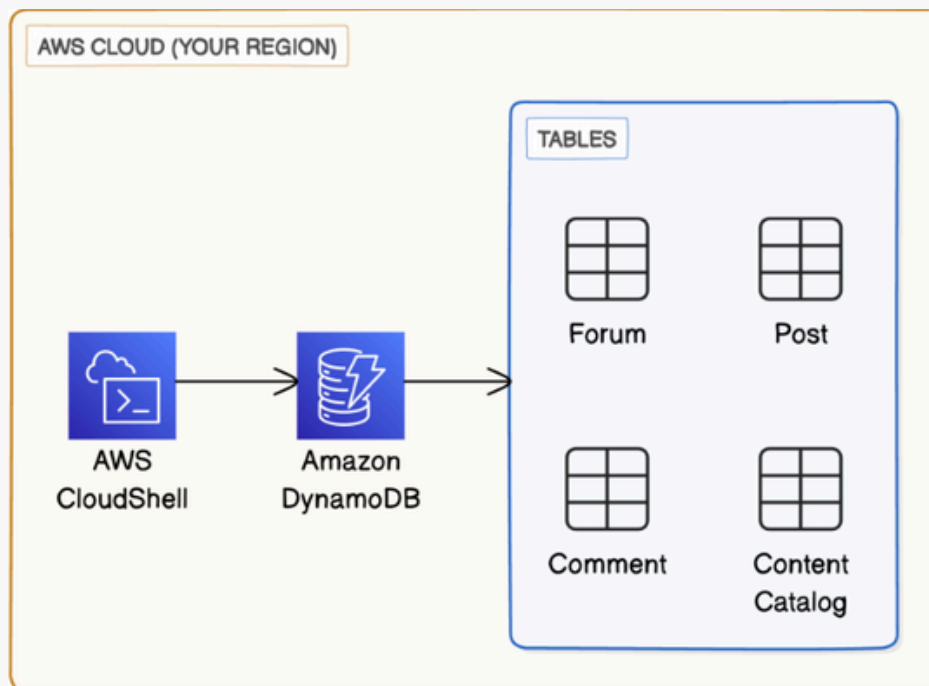


Nikko Cajayon
Community Lead



Cloud Project:

Query Data with AWS DynamoDB using AWS CloudShell



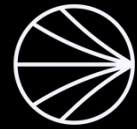


TABLE OF CONTENTS

1

INTRODUCTION

Introduction
System Architecture
Tools and services

2

PROVISION DYNAMODB TABLES USING CLOUDSHELL

Understanding DynamoDb as Non-relational Database
Defference between NoSQL and SQL

3

LOADING DATA INTO OUR TABLE

Manually load data
Load Data using AWS CLI

4

RUN YOUR A QUERY WITH THE CONSOLE

Query Data using DyndamoDB Console

5

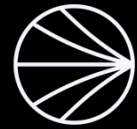
RUN QUERIES WITH AWS CLI

Query Data using AWS CloudShell's CLI

6

SET UP A TRANSACTION

transact-write-items command.

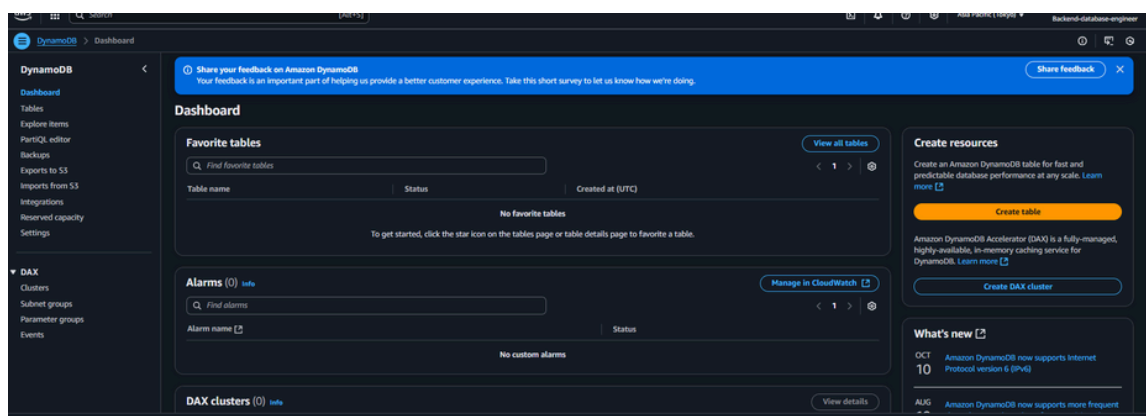


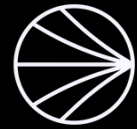
1

INTRODUCTION

This is the last part of my AWS Database project series, we started from Visualizing a relational database which give us the foundational understanding of database services in AWS environment work, following with our second project series Aurora Database with Elastic Compute Cloud (EC2) which helps us to grasp the concept of cloud architecture. after this we dive into a more complex project where we connect our Aurora Database to EC2 webserver hosting our web application giving as the skill of security and networks best practices. In preparation for this project, we also created a project on how we are going to load data to our AWS Dynamo Database using the tradition method and automated method using AWS CloudShell.

In this Final project, I will be showing how we are going to provision, load, and query our AWS Dynamo Database. this will help us understand how powerful the pre-installed Command Line Interface of AWS CloudShell, and how it can help us to seamless interact with our Database.





TOOLS AND SERVICES

AWS IAM (Identity Access Management)

- In AWS, an IAM user is a person or a computer that can do things on the AWS cloud. It's best practice for account security to use an IAM user with limited access for everyday tasks instead of the root user. In this project, you will log in with your IAM user to access your AWS account and perform actions related to DynamoDB.

Amazon DynamoDB

- This is a fast and flexible NoSQL database service for all applications that need consistent, single-digit millisecond latency at any scale. In this project, you'll be.

AWS CLI (Command Line Interface)

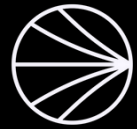
- This is a unified tool to manage your AWS services from the command line. You'll use the AWS CLI to interact with your DynamoDB table, such as uploading data and querying it.

AWS CloudShell:

- This is a browser-based shell that makes it easy to securely manage, explore, and interact with your AWS resources. You'll be using CloudShell to execute AWS CLI commands without needing to install the CLI locally..



Nikko Cajayon
NextWork Community Lead

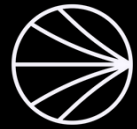


IAM (IDENTITY ACCESS MANAGEMENT) ACCOUNT

IAM is like a special account you create within your AWS account for different people or applications. It's a way to give them specific permissions to access and manage AWS resources without sharing your main account credentials. Think of it as creating separate logins for your team members, each with limited access to only what they need.

IAM users are crucial for security and access control in AWS. Here's a breakdown:

- **Security:** IAM users prevent you from sharing your root account credentials, which have unrestricted access to everything. This reduces the risk of unauthorized access and potential damage.
- **Granular Permissions:** You can assign specific permissions to each IAM user, controlling which AWS services and resources they can access and what actions they can perform.
- **User Management:** IAM allows you to easily manage users, groups, and roles within your AWS account. You can create groups with common permissions and add users to those groups for simplified administration.
- **Access Keys:** IAM users can be assigned access keys for programmatic access to AWS services via the AWS CLI, SDKs, or APIs. These keys should be managed securely.



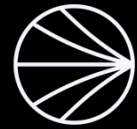
AWS DYNAMO DATABASE

WHAT IS DYNAMODB?

fully managed, serverless, NoSQL database service provided by AWS. Think of it as a super flexible digital filing cabinet that can store and quickly retrieve massive amounts of data without you having to worry about managing the underlying

LEARN MORE ABOUT DYNAMODB

- **NoSQL Database:** Unlike traditional relational databases, DynamoDB doesn't use SQL. It uses key-value and document data models, making it highly flexible and scalable.
- **Serverless:** You don't need to provision, patch, or manage servers. AWS handles all the administrative tasks, allowing you to focus on your application.
- **Scalability:** DynamoDB can automatically scale up or down to handle your workload, ensuring consistent performance even during peak times.
- **Performance:** It's designed for low-latency access, providing fast and predictable performance at any scale.
- **Data Model:** Data is stored in tables, which contain items. Each item is a collection of attributes. You define a primary key (partition key and optionally a sort key) to uniquely identify each item.



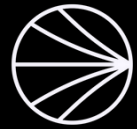
UNDERSTANDING NON-RELATIONAL DATABASES

NON-RELATIONAL DATABASES?

Non-relational databases, also known as NoSQL databases is a type of database that doesn't use the traditional table format of rows and columns. Instead, it uses other ways to organize data, like storing it in documents, key-value pairs, or graphs. This makes it more flexible for handling different types of information.

DIFFERENT OF NOSQL TO SQL DATABASE

- **Data Model:** Instead of tables, NoSQL databases use various data models such as document, key-value, graph, or column-family. This allows for storing unstructured or semi-structured data.
- **Schema Flexibility:** NoSQL databases often have dynamic schemas, meaning that the structure of the data can vary from one entry to another. This is useful when dealing with evolving data requirements.
- **Scalability:** NoSQL databases are designed to scale horizontally, distributing data across many machines to handle large volumes of data and high traffic.
- **Query Language:** Unlike relational databases that use SQL, NoSQL databases have their own query languages or methods, which are often tailored to the specific data model.



AWS CLOUDSHELL AND AWS CLI

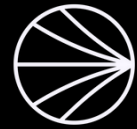
AWS CLOUDSHELL

AWS CloudShell is shell in your AWS Management Console, which means it's a space for you to run code! The awesome thing about AWS CloudShell is that it already has AWS CLI pre-installed.

AWS COMMAND LINE INTERFACE (CLI)

A software that lets you create, delete and update AWS resources with commands instead of clicking through your console.

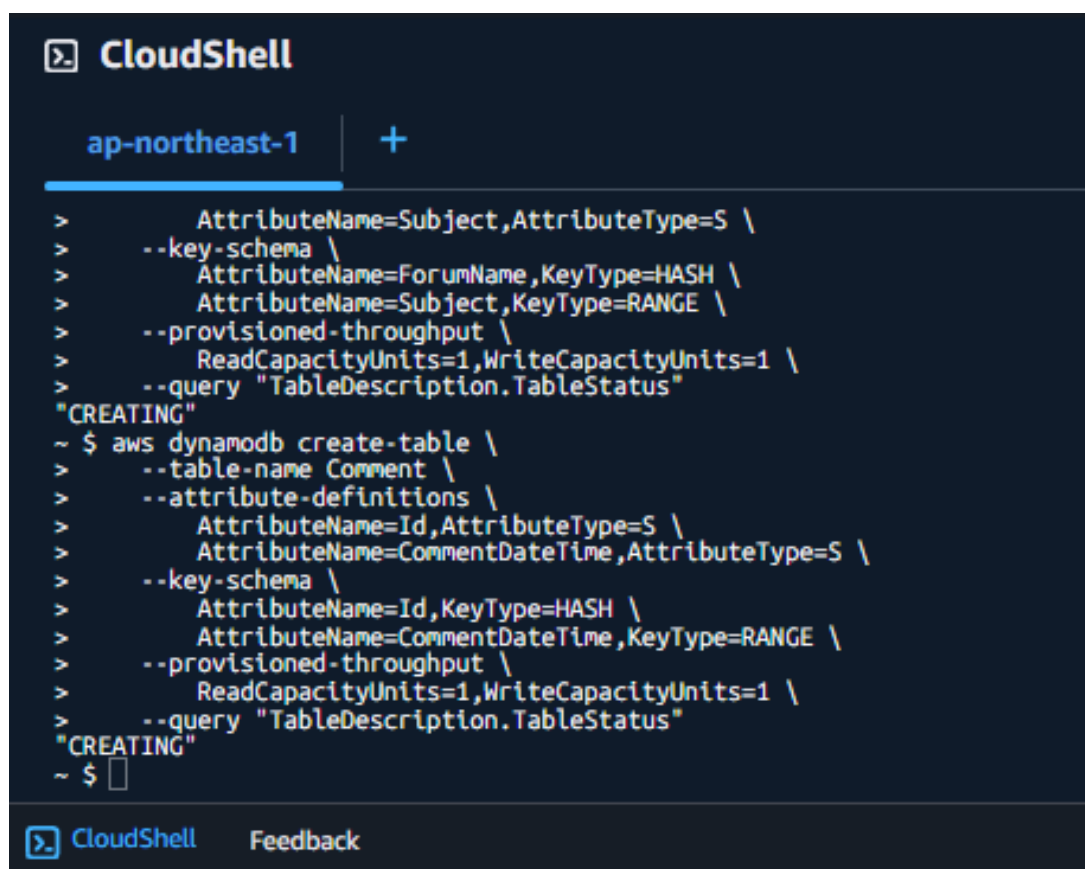
You usually have to install AWS CLI into your computer to use it, but in our case, CloudShell already has CLI installed for us



2

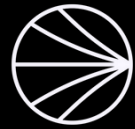
PROVISION DYNAMODB TABLES USING CLOUDSHELL

In this part, we will be running some commands using AWS CLI of AWS CloudShell to provision our DynamoDB Tables in which we will later ingest our data



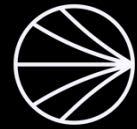
```
CloudShell
ap-northeast-1 +
> AttributeName=Subject,AttributeType=S \
> --key-schema \
> AttributeName=ForumName,KeyType=HASH \
> AttributeName=Subject,KeyType=RANGE \
> --provisioned-throughput \
> ReadCapacityUnits=1,WriteCapacityUnits=1 \
> --query "TableDescription.TableStatus"
"CREATING"
~ $ aws dynamodb create-table \
> --table-name Comment \
> --attribute-definitions \
> AttributeName=Id,AttributeType=S \
> AttributeName=CommentDateTime,AttributeType=S \
> --key-schema \
> AttributeName=Id,KeyType=HASH \
> AttributeName=CommentDateTime,KeyType=RANGE \
> --provisioned-throughput \
> ReadCapacityUnits=1,WriteCapacityUnits=1 \
> --query "TableDescription.TableStatus"
"CREATING"
~ $
```

This script includes commands to create four new tables in AWS DynamoDB, each with specific attributes and settings.

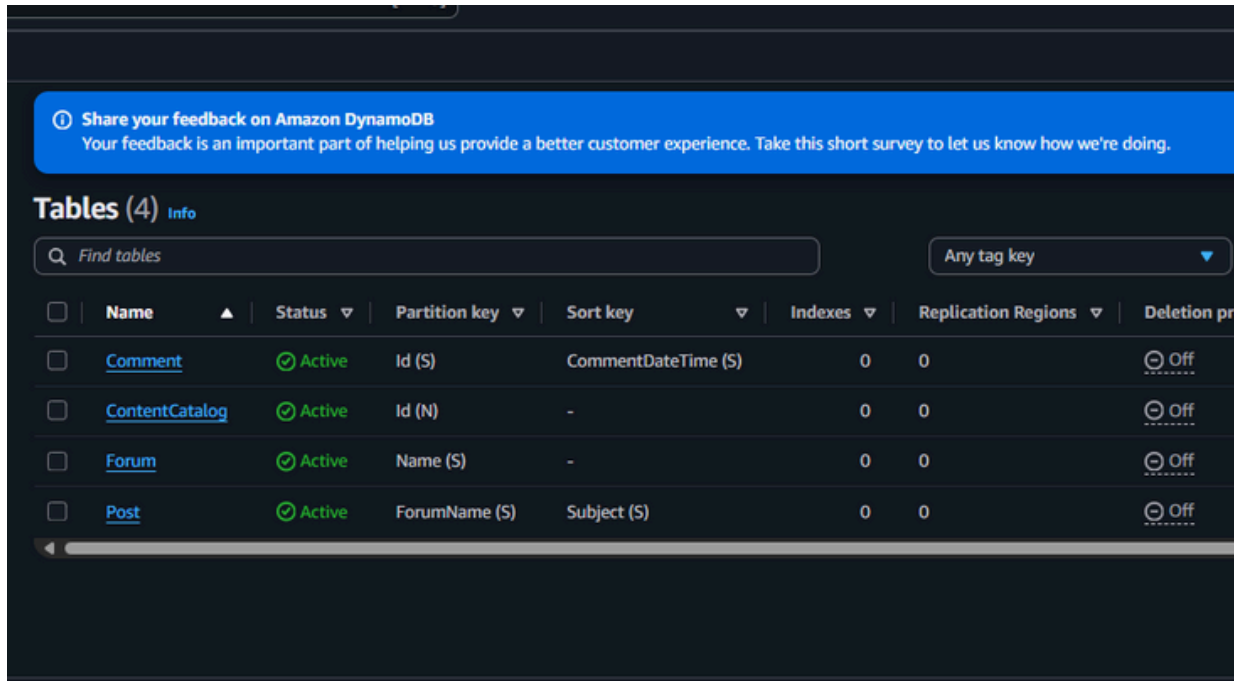


SCRIPT FOR AWS CLOUDSHELL TO PROVISION DYNAMODB TABLES

```
aws dynamodb create-table \  
  --table-name ContentCatalog \  
  --attribute-definitions \  
    AttributeName=Id,AttributeType=N \  
  --key-schema \  
    AttributeName=Id,KeyType=HASH \  
  --provisioned-throughput \  
    ReadCapacityUnits=1,WriteCapacityUnits=1 \  
  --query "TableDescription.TableStatus"  
aws dynamodb create-table \  
  --table-name Forum \  
  --attribute-definitions \  
    AttributeName=Name,AttributeType=S \  
  --key-schema \  
    AttributeName=Name,KeyType=HASH \  
  --provisioned-throughput \  
    ReadCapacityUnits=1,WriteCapacityUnits=1 \  
  --query "TableDescription.TableStatus"  
aws dynamodb create-table \  
  --table-name Post \  
  --attribute-definitions \  
    AttributeName=ForumName,AttributeType=S \  
    AttributeName=Subject,AttributeType=S \  
  --key-schema \  
    AttributeName=ForumName,KeyType=HASH \  
    AttributeName=Subject,KeyType=RANGE \  
  --provisioned-throughput \  
    ReadCapacityUnits=1,WriteCapacityUnits=1 \  
  --query "TableDescription.TableStatus"  
aws dynamodb create-table \  
  --table-name Comment \  
  --attribute-definitions \  
    AttributeName=Id,AttributeType=S \  
    AttributeName=CommentDateTime,AttributeType=S \  
  --key-schema \  
    AttributeName=Id,KeyType=HASH \  
    AttributeName=CommentDateTime,KeyType=RANGE \  
  --provisioned-throughput \  
    ReadCapacityUnits=1,WriteCapacityUnits=1 \  
  --query "TableDescription.TableStatus"
```



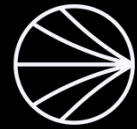
TABLES IN DYNAMODB



The screenshot shows the Amazon DynamoDB console interface. At the top, there is a blue banner with a feedback survey link. Below this, the 'Tables (4)' section is visible, featuring a search bar and a dropdown menu for 'Any tag key'. A table lists four DynamoDB tables: 'Comment', 'ContentCatalog', 'Forum', and 'Post'. Each row includes a checkbox, the table name, its status (all are 'Active'), its partition key, its sort key, the number of indexes, the number of replication regions, and a deletion protection status (all are 'Off').

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection
☐	Comment	Active	Id (S)	CommentDateTime (S)	0	0	Off
☐	ContentCatalog	Active	Id (N)	-	0	0	Off
☐	Forum	Active	Name (S)	-	0	0	Off
☐	Post	Active	ForumName (S)	Subject (S)	0	0	Off

- **ContentCatalog Table:** This table stores information about various content items, such as projects and videos. It uses a numeric attribute called Id as its partition key.
- **Forum Table:** This table stores information about different forums or discussion groups. It uses a string attribute called Name as its partition key.
- **Post Table:** This table stores information about individual posts within a forum. It uses ForumName as the partition key and Subject as the sort key. The sort key allows for ordering posts within each forum.
- **Comment Table:** This table stores comments made on posts. It uses Id as the partition key and CommentDateTime as the sort key. The sort key allows for ordering comments based on when they were posted.



3 LOADING DATA INTO OUR TABLE

In this part, We are going to load data into our DynamoDB Tables using AWS CloudShell CLI. and try to run some actions like view, delete, and write. this will help us understand how we can seamlessly connect with Database using CloudShell's CLI.

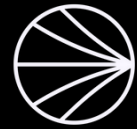
curl -O

https://storage.googleapis.com/nextwork_course_resources/courses/aws/AWS%20Project%20People%20projects/Project%3A%20Query%20Data%20with%20DynamoDB/nextworksampladata.zip

unzip nextworksampladata.zip

cd nextworksampladata

```
CloudShell
ap-northeast-1 +
replace __MACOSX/nextworksampladata/._ContentCatalog.json? [y]es, [n]o, [A]ll, [N]one, [r]ename: -y
error: invalid response [-y]
replace __MACOSX/nextworksampladata/._ContentCatalog.json? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: __MACOSX/nextworksampladata/._ContentCatalog.json
replace nextworksampladata/Comment.json? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: nextworksampladata/Comment.json
replace __MACOSX/nextworksampladata/._Comment.json? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: __MACOSX/nextworksampladata/._Comment.json
replace nextworksampladata/Forum.json? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: nextworksampladata/Forum.json
replace __MACOSX/nextworksampladata/._Forum.json? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: __MACOSX/nextworksampladata/._Forum.json
replace nextworksampladata/Post.json? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: nextworksampladata/Post.json
replace __MACOSX/nextworksampladata/._Post.json? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: __MACOSX/nextworksampladata/._Post.json
~ $ y
-bash: y: command not found
~ $ y
-bash: y: command not found
~ $ ls
__MACOSX  nextworksampladata  nextworksampladata.zip
~ $ cd nextworksampladata
nextworksampladata $ ls
Comment.json  ContentCatalog.json  'dynamodb delete-table --table-name ContentCatalog'  Forum.json  Post.json
nextworksampladata $
```

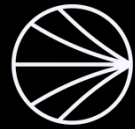



CLOUDSHELL'S CLI QUERY DATA

see how in just a single line of code we were able to view what's inside of our database, this is how CloudShell's CLI works it allows you to manage and oversee your AWS Resources without the need of checking and Database Console.

```
MACOSX nextworksampladata nextworksampladata.zip
~ $ cd nextworksampladata
nextworksampladata $ ls
Comment.json ContentCatalog.json 'dynamodb delete-table --table-name ContentCatalog' Forum.json
nextworksampladata $ cat Forum.json
{
  "Forum": [
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "I have a question"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "2"},
          "Comments": {"N": "4"},
          "Views": {"N": "1000"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "Events"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "1"}
        }
      }
    }
  ]
}
nextworksampladata $
```

CloudShell Feedback



CLOUDSHELL'S CLI LOAD DATA

Now let's try to run some few codes to check if we can load some new data in our tables.

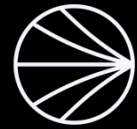
```
aws dynamodb batch-write-item --request-items file://ContentCatalog.json
```

```
aws dynamodb batch-write-item --request-items file://Forum.json
```

```
aws dynamodb batch-write-item --request-items file://Post.json
```

```
aws dynamodb batch-write-item --request-items file://Comment.json
```

```
CloudShell
ap-northeast-1 +
{
  "PutRequest": {
    "Item": {
      "Name": {"S": "I have a question"},
      "Category": {"S": "NextWorkCommunity"},
      "Posts": {"N": "2"},
      "Comments": {"N": "4"},
      "Views": {"N": "1000"}
    }
  },
  "PutRequest": {
    "Item": {
      "Name": {"S": "Events"},
      "Category": {"S": "NextWorkCommunity"},
      "Posts": {"N": "1"}
    }
  }
}
nextworksampladata $ aws dynamodb batch-write-item --request-items file://ContentCatalog.json
{
  "UnprocessedItems": {}
}
nextworksampladata $
nextworksampladata $ aws dynamodb batch-write-item --request-items file://Forum.json
{
  "UnprocessedItems": {}
}
nextworksampladata $
nextworksampladata $ aws dynamodb batch-write-item --request-items file://Post.json
{
  "UnprocessedItems": {}
}
nextworksampladata $
nextworksampladata $ aws dynamodb batch-write-item --request-items file://Comment.json
{
  "UnprocessedItems": {}
}
nextworksampladata $
```



CLOUDSHELL'S CLI LOAD DATA

And there you go, we successfully loaded some data inside of our ContentCatalog table

Table: ContentCatalog - Items returned (6)
Scan started on October 14, 2025, 11:58:52

☐	Id (Number) ▾	Authors ▾	ContentType ▾	Difficulty ▾	Price ▾
☐	3	[{"S": "NextWork"}]	Project	Easy peasy	0
☐	2	[{"S": "NextWork"}]	Project	Easy peasy	0
☐	203		Video		0
☐	202		Video		0
☐	201		Video		0
☐	1	[{"S": "Natasha"}]	Project	Easy peasy	0

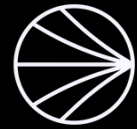
WHAT AM I SEEING?

The data you loaded when you ran aws dynamodb batch-write-item is here!

We can see now that the table has a partition key of Id (the very first column!), and there are 6 items in the table.

WHAT IS A PARTITION KEY?

Partition keys are like tags that DynamoDB uses to organise the table's data. When you search for an item in your table, DynamoDB will need its partition key!



FLEXIBILITY OF NOSQL

let's try to have some comparison between the two tables inside of our database.

VIDEO TABLE

Attributes Add new attribute ▼

Attribute name	Value	Type	
☐ Id - Partition key	203	Number	
ContentType	Video	String	<button>Remove</button>
Price	0	Number	<button>Remove</button>
☐ Services	Insert a field ▼	List	<button>Remove</button>
Title	AWS x Databases project: Web App with Aurora Database	String	<button>Remove</button>
URL	https://youtube.com/live/PCyE_IU6rrU	String	<button>Remove</button>
VideoType	Live Project Demo	String	<button>Remove</button>

Cancel Save Save and close

PROJECT TABLE

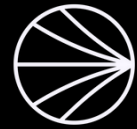
Attributes Add new attribute ▼

Attribute name	Value	Type	
☐ Id - Partition key	1	Number	
☐ Authors	Insert a field ▼	List	<button>Remove</button>
ContentType	Project	String	<button>Remove</button>
Difficulty	Easy peasy	String	<button>Remove</button>
Price	0	Number	<button>Remove</button>
ProjectCategory	Storage	String	<button>Remove</button>
Published	☒ True ☐ False	Boolean	<button>Remove</button>
Title	Host a Website on Amazon S3	String	<button>Remove</button>
URL	aws-host-a-website-on-s3	String	<button>Remove</button>

Cancel Save Save and close

ARE ALL THE ATTRIBUTES IN THE PREVIOUS ITEM HERE?

- Nope! For example, we don't see Difficulty or ProjectCategory here.
- This is the reason why DynamoDB is known for its flexibility - every single item can have their own set of attributes. Just because StudentsComplete is an attribute in the first item (with Id 1), that doesn't mean there's a StudentsComplete attribute in the other items in this table.



4

RUN YOUR A QUERY WITH THE CONSOLE

In this step, I am going to In this Step we will now run a code to query our database' tables using our CloudShell's CLI, and find a data using partition and sort keys.

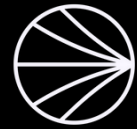
The screenshot shows the AWS ContentCatalog console interface. At the top, there's a header with 'ContentCatalog', 'Autopreview' toggle, and 'View table details' button. Below this is a section 'Scan or query items' with 'Scan' and 'Query' radio buttons. The 'Query' option is selected. Under 'Select a table or index', 'Table - ContentCatalog' is chosen. 'Select attribute projection' is set to 'All attributes'. The 'Partition key: Id' is set to '201'. There are 'Run' and 'Reset' buttons. A green status bar indicates 'Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 0.5'. Below this, a table titled 'Table: ContentCatalog - Items returned (1)' shows the query result. The table has columns: Id (Number), ContentType, Price, Services, Title, URL, and VideoType. The single row shows Id: 201, ContentType: Video, Price: 0, Services: [{"S": "Am..."}, Title: AWS Relati..., URL: https://you..., and VideoType: Educational.

Id (Number)	ContentType	Price	Services	Title	URL	VideoType
201	Video	0	[{"S": "Am..."}]	AWS Relati...	https://you...	Educational

WHAT IS A PARTITION KEY?

Think of a partition key as the filter that DynamoDB will use to split up and find data.

Partition key values don't have to be unique. For example if "Color" is a partition key, items can share partition key values like "Blue", "Green", "Red" and more



SCAN THROUGH THE CONSOLE

In this part, we will be using the DynamoDB console to query our Database' tables to check the content and items within.

Comment Autopreview View table details

▼ Scan or query items

☐ Scan ☒ Query

Select a table or index: Table - Comment

Select attribute projection: All attributes

Partition key: Id

Enter partition key value

ⓘ The partition key filter cannot be empty.

Sort key: CommentDateTime

Greater than Enter sort key value ☐ Sort descending

▼ Filters - optional

Attribute name	Condition	Type	Value	
PostedBy	Equal to	String	User Abdulrahman	Remove

Add filter

Run Reset

Completed - Items returned: 2 - Items scanned: 2 - Efficiency: 100% - RCUs consumed: 0.5

Table: Comment - Items returned (2)

Query started on October 14, 2025, 12:28:17

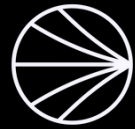
Actions Create item

THERE'S AN ERROR!

That's right - you have to use the Id (Partition key) when you query items.

This teaches us two things:

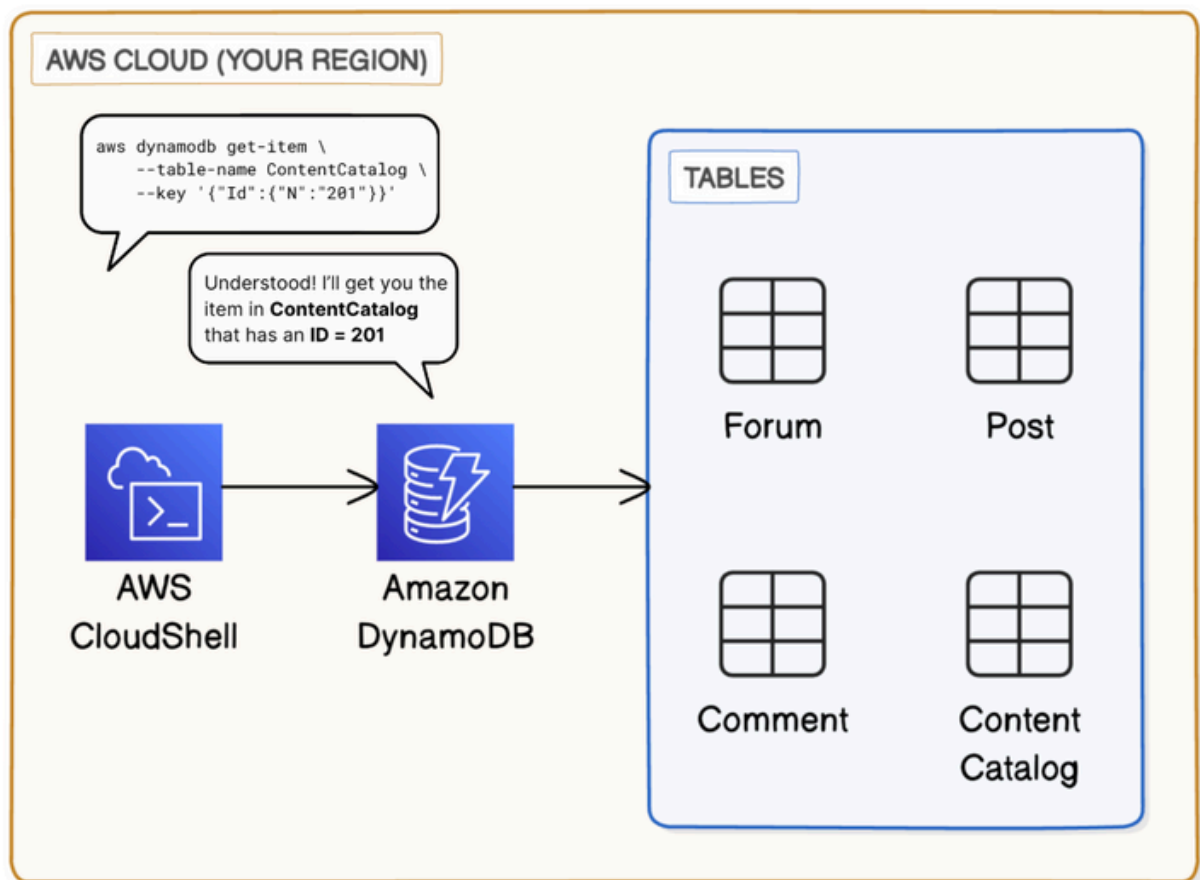
1. Data modelling is sooo important - data engineers think very carefully about what queries they should prepare and design for before they upload any data!
2. We probably wouldn't have this kind of issue if we could use SQL, so this is a situation where it would be beneficial to use a relational database!



5

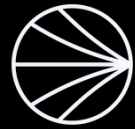
RUN QUERIES WITH AWS CLI

In this part we will be running some code to query our database using AWS Command Line Interface CLI



COMMAND

```
aws dynamodb get-item \  
  --table-name ContentCatalog \  
  --key '{"Id":{"N":"201"}}'
```



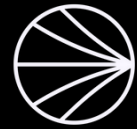
```
CloudShell
ap-northeast-1 +
~ $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"201"}}'
~ $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"201"}}'
{
  "Item": {
    "Title": {
      "S": "AWS Relational vs Non Relational Databases in 6 minutes"
    },
    "Price": {
      "N": "0"
    },
    "VideoType": {
      "S": "Educational"
    },
    "ContentType": {
      "S": "Video"
    },
    "Services": {
      "L": [
        {
          "S": "Amazon RDS"
        },
        {
          "S": "Amazon DynamoDB"
        }
      ]
    },
    "Id": {
      "N": "201"
    }
  },
  ...skipping...
}
```

WHAT DOES THIS COMMAND DO?

`aws dynamodb get-item` is the AWS CLI command to get a single item from a DynamoDB table.

`--table-name ContentCatalog` tells DynamoDB that the item belongs in the table called ContentCatalog.

`--key '{"Id":{"N":"201"}}'` tells DynamoDB that the item has the ID "201". The `{"N":}` means the ID is a number (N for Number).

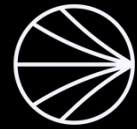


```
aws dynamodb get-item \  
  --table-name ContentCatalog \  
  --key '{"Id":{"N":"101"}}' \  
  --consistent-read \  
  --projection-expression "Title, ContentType, Services" \  
  --return-consumed-capacity TOTAL
```

```
CloudShell  
ap-northeast-1 +  
...skipping...  
{  
  "Item": {  
    "Title": {  
      "S": "AWS Relational vs Non Relational Databases in 6 minutes"  
    },  
    "Price": {  
      "N": "0"  
    },  
    "VideoType": {  
      "S": "Educational"  
    },  
    "ContentType": {  
      "S": "Video"  
    },  
    "Services": {  
      "L": [  
        {  
          "S": "Amazon RDS"  
        },  
        {  
          "S": "Amazon DynamoDB"  
        }  
      ]  
    },  
    "Id": {  
      "N": "201"  
    },  
    "URL": {  
      "S": "https://youtu.be/SzvdEf9y0eA"  
    }  
  }  
}  
(END)
```

This command is just like the `get-item` command we ran before, We're adding extra query options that tells DynamoDB exactly how we want it to get the item:

- **--consistent-read** : you want a strongly consistent read.
- **--projection-expression**: you only want to know some of the item's attributes.
- **--return-consumed-capacity** : you want to know how much capacity was consumed by the request.

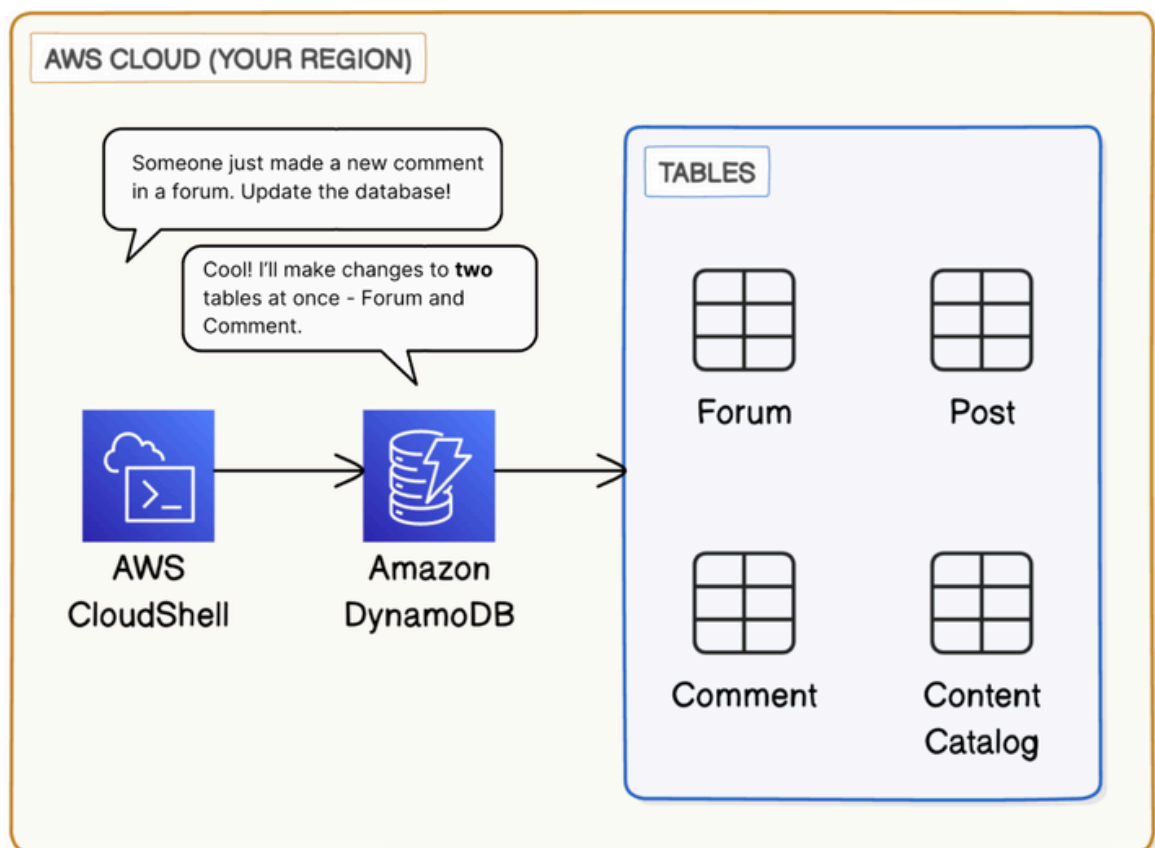


6

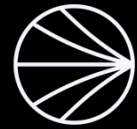
SET UP A TRANSACTION

For our final part of this project, you might notice that our tables contain lots of related data!

- Each Forum contains multiple Posts.
- Each Post contains multiple Comments.



In this final part, we will be handling related data across DynamoDB tables, then we will update tables in a single AWS CLI Command.



SET UP A TRANSACTION

```
aws dynamodb transact-write-items --client-request-token TRANSACTION1 --transact-items '[
{
  "Put": {
    "TableName": "Comment",
    "Item": {
      "Id": {"S": "Events/Do a Project Together - NextWork Study Session"},
      "CommentDateTime": {"S": "2024-9-27T17:47:30Z"},
      "Comment": {"S": "Excited to attend!"},
      "PostedBy": {"S": "User Connor"}
    }
  },
  "Update": {
    "TableName": "Forum",
    "Key": {"Name": {"S": "Events"}},
    "UpdateExpression": "ADD Comments :inc",
    "ExpressionAttributeValues": { ":inc": {"N": "1"} }
  }
}
]'
```

This command is a transaction, which is a group of operations that all have to succeed - if any of the operations in the group fails, none of the changes get applied. This makes sure that any change to your database is consistent across all your tables!

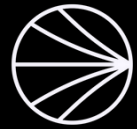
```
aws dynamodb get-item \
--table-name Forum \
--key '{"Name": {"S": "Events"}}'
```

I ran a transaction using the `aws dynamodb transact-write-items` command.

This command grouped two separate actions into a single "all-or-nothing" operation:

1. Put: It added a new comment to the Comment table.
2. Update: It incremented the comment counter in the Forum table.

This guarantees that both actions succeed together or both fail, preventing data inconsistency. The final `get-item` command was then used to verify that the comment count was successfully updated.



PROJECT SUMMARY: AWS DYNAMODB PROJECT SERIES

This project series documents the complete lifecycle of building, populating, and interacting with a NoSQL database using Amazon DynamoDB. Executed entirely through the AWS Command Line Interface (CLI), this project provided a practical, hands-on demonstration of core cloud database engineering principles, from initial design to advanced data manipulation.

Next, we focused on retrieving the data. We practiced fetching specific items with `get-item` and querying for related records, learning that a partition key is always required. Finally, we demonstrated how to keep data in sync across multiple tables by using a single, all-or-nothing transaction command to perform a complex update.

The hands-on experience gained through this project series has been invaluable, and I am excited to implement these skills and concepts in my future large-scale projects.



Nikko Cajayon

NextWork Community Lead
Director for SkillBuilder
CTO of AWS Cloud Club

Here at NextWork We are Transforming
the Way World Learns